Deployment Instructions (CLI) — Inventory Planning Agent

Each deployment gets its own versioned resource group and Container Apps environment. Only the ACR (`inventoryacr2`) is shared — you push a new image tag each time.

> **Important:** All `az containerapp` commands must be run as a single line in zsh. Multiline with `\` causes "command not found" errors in interactive shells.

Set Variables (run these at the start of every session)

```
VERSION=v5                          # increment for each new deployment

# Shared (never change)
SUBSCRIPTION=demoproj
ACR=inventoryacr2.azurecr.io
ACR_NAME=inventoryacr2
API_KEY=your-secret-key-here        # pick once, reuse across all deployments
GROQ_API_KEY=<Add your api key here>     # from console.groq.com

# Per-deployment (derived from VERSION)
RG=inventory-rg-$VERSION
ENV=inventory-env-$VERSION
STORAGE=inventorysa$VERSION         # lowercase alphanumeric only, max 24 chars
SHARE=inventorydb
LOCATION=eastus
```

Step 1 — Login

```
az login
az account set --subscription $SUBSCRIPTION
```

Step 2 — Create Resource Group

```
az group create --name $RG --location $LOCATION
```

Step 3 — Build and Push Images to ACR

```
az acr login --name $ACR_NAME

# Build from repo root — Dockerfiles copy db/schema.sql so context must be .
docker build --platform linux/amd64 -f backend/Dockerfile -t
$ACR/inventory-backend:$VERSION .
docker build --platform linux/amd64 -f frontend/Dockerfile -t
$ACR/inventory-frontend:$VERSION .

docker push $ACR/inventory-backend:$VERSION
docker push $ACR/inventory-frontend:$VERSION
```

Step 4 — Create Storage Account and File Share

```
az storage account create --name $STORAGE --resource-group $RG --location
$LOCATION --sku Standard_LRS

az storage share create --name $SHARE --account-name $STORAGE

STORAGE_KEY=$(az storage account keys list --account-name $STORAGE
--resource-group $RG --query "[0].value" -o tsv)
```

Step 5 — Create Container Apps Environment

```
az containerapp env create --name $ENV --resource-group $RG --location
$LOCATION
```

Step 6 — Link File Share to Environment

```
az containerapp env storage set --name $ENV --resource-group $RG
--storage-name inventorydbstorage --azure-file-account-name $STORAGE
--azure-file-account-key $STORAGE_KEY --azure-file-share-name $SHARE
--access-mode ReadWrite
```

Step 7 — Deploy Backend (with persistent volume)

Part 7a — Create the app (no volume yet)

```
ACR_PASSWORD=$(az acr credential show --name $ACR_NAME --query
"passwords[0].value" -o tsv)

az containerapp create --name inventory-backend --resource-group $RG
--environment $ENV --image $ACR/inventory-backend:$VERSION --target-port 8000
```

```
--ingress internal --registry-server $ACR --registry-username $ACR_NAME
--registry-password "$ACR_PASSWORD" --secrets "groq-key=$GROQ_API_KEY"
"api-key=$API_KEY" --env-vars "GROQ_API_KEY=secretref:groq-key"
"API_KEY=secretref:api-key" "DB_PATH=/app/db/inventory.db" --cpu 1.0 --memory
2Gi
```

Part 7b — Patch in the volume mount via REST API

```
SUB_ID=$(az account show --query id -o tsv)

cat > /tmp/volume_patch.json << EOF
{
  "properties": {
    "template": {
      "volumes": [
        {
          "name": "dbvolume",
          "storageType": "AzureFile",
          "storageName": "inventorydbstorage"
        }
      ],
      "containers": [
        {
          "name": "inventory-backend",
          "image": "$ACR/inventory-backend:$VERSION",
          "env": [
            {"name": "GROQ_API_KEY", "secretRef": "groq-key"},
            {"name": "API_KEY",      "secretRef": "api-key"},
            {"name": "DB_PATH",      "value": "/app/db/inventory.db"}
          ],
          "resources": {"cpu": 1.0, "memory": "2Gi"},
          "volumeMounts": [
            {"volumeName": "dbvolume", "mountPath": "/app/db"}
          ]
        }
      ]
    }
  }
}
EOF

az rest --method PATCH --url
"https://management.azure.com/subscriptions/$SUB_ID/resourceGroups/$RG/provid
ers/Microsoft.App/containerApps/inventory-backend?api-version=2023-05-01"
--headers "Content-Type=application/json" --body @/tmp/volume_patch.json
```

Step 8 — Get Backend Internal URL

```
BACKEND_FQDN=$(az containerapp show -n inventory-backend -g $RG --query
"properties.configuration.ingress.fqdn" -o tsv)

echo "Backend URL: https://$BACKEND_FQDN"
```

> The URL will contain `.internal.` — this is correct and expected. The backend is only reachable from within the Container Apps environment. **You cannot curl it from your local machine.** Skip Step 9 and go straight to Step 10.

Step 9 — Validate Backend (optional, Azure Cloud Shell only)

> Run these from **Azure Cloud Shell** (portal.azure.com → Cloud Shell icon top right). Replace values with your actual BACKEND_FQDN and API_KEY.

```
# Health check (GET)
curl https://<your-backend-fqdn>/health -H "X-API-Key: your-secret-key-here"
# Expected: {"status": "ok"}

# Chat test
curl -X POST https://<your-backend-fqdn>/chat -H "Content-Type:
application/json" -H "X-API-Key: your-secret-key-here" -d '{"query": "Should
I reorder SKU001?", "sku_id": "SKU001"}'
```

Step 10 — Deploy Frontend (public)

```
az containerapp create --name inventory-frontend --resource-group $RG
--environment $ENV --image $ACR/inventory-frontend:$VERSION --target-port
7860 --ingress external --registry-server $ACR --registry-username $ACR_NAME
--registry-password "$ACR_PASSWORD" --env-vars
"BACKEND_URL=https://$BACKEND_FQDN" "API_KEY=$API_KEY" "PORT=7860"
--min-replicas 1
```

Set backend min replicas too (prevents scale-to-zero which drops connections):

```
az containerapp update -n inventory-backend -g $RG --min-replicas 1
```

Step 10c — Cost Optimisation (optional)

By default both containers run 24/7 (`--min-replicas 1`) costing ~$12 per 3 days. To reduce cost to under $1, scale to zero when idle:

```
az containerapp update -n inventory-backend -g $RG --min-replicas 0
az containerapp update -n inventory-frontend -g $RG --min-replicas 0
```

Trade-off: The first request after a period of inactivity will take 10-20 seconds (cold start) while the containers spin back up. Subsequent requests are fast. Recommended for low-traffic or demo deployments.

Step 11 — Get Frontend Public URL

```
az containerapp show -n inventory-frontend -g $RG --query
"properties.configuration.ingress.fqdn" -o tsv
```

Open the URL in a browser — the Gradio UI should load. Test by entering `SKU001` in the SKU panel and asking “Should I reorder?”

Redeployment

Full redeployment (new VERSION, fresh infrastructure)

Change `VERSION=v2` in the variables block and re-run all steps 1–11. Delete the old resource group once the new one is confirmed working:

```
az group delete --name inventory-rg-v1 --yes --no-wait
```

Frontend-only update (code change, same infrastructure)

```
docker build --platform linux/amd64 -f frontend/Dockerfile -t
$ACR/inventory-frontend:$VERSION .
docker push $ACR/inventory-frontend:$VERSION
az containerapp update -n inventory-frontend -g $RG --image
$ACR/inventory-frontend:$VERSION
```

Backend-only update (code change, same infrastructure)

```
docker build --platform linux/amd64 -f backend/Dockerfile -t
$ACR/inventory-backend:$VERSION .
docker push $ACR/inventory-backend:$VERSION
az containerapp update -n inventory-backend -g $RG --image
$ACR/inventory-backend:$VERSION
```

Quick Reference

Set variables block first before running any of these.

What	Command
List inventory resource groups	`az group list --query "[?starts_with(name,'inventory-rg')]" -o table`

What	Command
Stream frontend logs	`az containerapp logs show -n inventory-frontend -g $RG --follow`
Stream backend logs	`az containerapp logs show -n inventory-backend -g $RG --follow`
Check system events	`az containerapp logs show -n inventory-frontend -g $RG --type system --tail 50`
Check running status	`az containerapp show -n inventory-backend -g $RG --query "properties.runningStatus" -o tsv`
List revisions	`az containerapp revision list -n inventory-backend -g $RG -o table`
Delete current deployment	`az group delete --name $RG --yes --no-wait`